Viva Script — ML: How the Models Classify

Sep 24, 2026 · @Sahil Khan Pathan

1. Opening (about 30 seconds)

Read the script as written, or use the bold lines as prompts. Lines in *[brackets]* are cues for what to show.

> "Our machine learning question is simple. Given one 60-second operating cycle of sensor data, what condition is each of four components in?
>
> We treat it as **four separate multiclass classification problems**: the cooler with 3 classes, the valve with 4, pump leakage with 3 and the accumulator with 4. Each component gets its own classifier.
>
> In the next few minutes I will explain how the models actually make a decision, how we chose them, how far their scores can be trusted, and exactly where they fail."

[Show ML deck, slide 2]

2. What happens when a cycle arrives

> "When a new cycle comes in, it passes through one fixed chain of steps. Every step was fitted on training data only and saved to disk, so the app never re-learns anything at prediction time."

Raw cycle (43,680 values) → 626 features → model's input columns → imputation + scaling → selection (valve, pump) → classifier → highest probability = prediction

Say each step in one line:

1. **Feature extraction** turns the raw signals into 626 named features, exactly as in training.
2. **Column alignment**: each model picks its own input columns by name, so column order can never silently break a prediction.
3. **Imputation and scaling**: gaps are filled with the training median, then each feature becomes a z-score using the training mean and standard deviation.
4. **Selection** (valve and pump models only): the saved correlation filter, mutual-information filter and tree-importance filter keep the same 40 features chosen during training.

5. **Classifier**: outputs one probability per class. The class with the highest probability is the prediction.
6. **Severity and system status**: each class maps to Normal, Degraded or Critical from the dataset documentation. The overall status is the worst of the four components.

> "The key point is that all these steps live inside one scikit-learn Pipeline. When we cross-validated, every step was re-fitted inside each training fold, so no test cycle ever influenced a scaler, a selector or a classifier."

3. How each classifier decides

The final models are two tree-ensemble families. Explain the single tree first, then the two ways of combining trees.

Component	Final model	Inputs reaching the model	Class weighting
Cooler	XGBoost	17 sensor means	none (classes balanced, ratio 1.01)
Valve	Random forest	40 selected features	balanced (ratio 3.13)
Pump leakage	XGBoost	40 selected features	balanced (ratio 2.48)
Accumulator	Random forest	all 553 features	balanced (ratio 2.03)

One decision tree.

> "A decision tree asks a series of yes/no questions about the features, for example: is the variation of pressure PS2 above some value? Each answer sends the cycle down a branch until it reaches a leaf. The leaf holds the class proportions of the training cycles that ended there, and those proportions are the tree's probabilities."

Random forest (valve, accumulator): many independent trees that vote.

> "A random forest grows 300 trees. Each tree sees a random bootstrap sample of the training cycles and, at every split, only a random subset of features. That makes the trees different from each other. To classify a cycle, every tree gives its class probabilities and the forest averages them. If most trees point to 'optimal valve behaviour', that class gets the highest averaged probability and wins."

Why it suits us: averaging many different trees reduces variance, captures feature interactions automatically and needs no scaling. With class weighting, rare fault classes count more when a tree chooses its splits.

XGBoost (cooler, pump): trees built one after another, each fixing the last one's mistakes.

> "Gradient boosting builds 300 shallow trees in sequence, each at most 5 levels deep. Each new tree is trained on the errors the ensemble is still making, and its output is added with a learning rate of 0.1. For a multiclass problem it keeps one running score per class. At the end, the scores are turned into probabilities with the softmax function."

Formula: P(class k) = e^(F_k) / (e^(F_1) + … + e^(F_K)), summed over all K classes, where F_k = the sum over 300 trees of 0.1 × (that tree's output for class k).

> "Here F_k is the summed score for class k, and h is the output of each small tree. XGBoost also penalises large trees and large leaf values, which keeps it from memorising noise."

Why not logistic regression or SVM? We trained them too. They only competed once they had engineered features, for example the valve went from 0.672 to 0.991 macro-F1 for logistic regression, and they never beat the tree ensembles. When scores tied, our rule picked the simpler option, which is why the cooler uses XGBoost on just 17 inputs.

4. Worked example: cycle 499

[Open MLapp, Predict tab, type 499, press Predict]

> "Let me score a real cycle. Cycle 499 was held out of training for the accumulator model, so it is a genuine unseen test for it."

Component	Recorded condition	Predicted	Confidence	Result
Cooler	3 % (close to total failure)	3 %	1.000	correct
Valve	90 % (small lag)	90 %	0.873	correct
Pump leakage	2 (severe)	2	0.998	correct
Accumulator	100 bar (severely reduced)	90 bar	0.493	**wrong**

> "The overall status is Critical, because the cooler is close to total failure. Three components are right with high confidence.
>
> The accumulator is wrong, and this is the interesting part. The forest gave 0.49 to 90 bar and 0.32 to the correct 100 bar. That is a margin of only 0.17, and no class is above one half. So the trees were split. The model did not fail confidently. It failed unsure, and the confidence number told us so."

[Switch to the Error analysis tab, accumulator selected]

> "The error-analysis view shows why. On the model's most important features, this cycle sits closer to the average 90-bar cycle than to the average 100-bar cycle. The two lowest pre-charge levels produce very similar pressure transients, so they overlap in feature space."

5. How we chose and evaluated the models

> "For every component we tried 4 model families on 3 feature sets, which is 12 configurations per component and 48 in total. Hyperparameters were held fixed, so any difference comes from the model type or the features, not from lucky tuning."

1. **Split once:** 80 % training (1,764 cycles), 20 % test (441 cycles), stratified so every class keeps its share.
2. **Choose on training data only:** 5-fold stratified cross-validation, ranked by macro-F1. Ties go to the simpler model and feature set.
3. **Test once:** the winner is refitted on all training data and scored exactly once on the untouched test set.
4. **Check for leakage:** blocked cross-validation holds out whole runs of 25 consecutive cycles.

> "We use **macro-F1**, not plain accuracy. It averages the F1 score of every class equally, so a model cannot score well by only getting the common 'healthy' class right."

Component	Random CV F1	Test F1	Blocked CV F1	Gap (random − blocked)
Cooler	0.998	1.000	0.995	0.003
Valve	0.995	1.000	0.984	0.011
Pump leakage	0.995	1.000	0.968	0.027
Accumulator	0.992	0.988	0.916	0.076

[Show ML deck, slide 5]

> "The blocked column is our honest number. Consecutive cycles are recorded seconds apart under identical settings, so they are near-duplicates. A random split puts those twins on both sides, and the model is partly tested on cycles it has effectively seen. Blocked validation removes that shortcut, and the accumulator drops by 7.6 points."

6. Model behaviour: errors, confidence and noise

Errors. Across all four models and 1,764 test predictions, there are only 5 errors. All five are accumulator cycles at 100 bar predicted as 90 bar.

> "The errors are conservative. The model says the accumulator is worse than it is, which triggers an early inspection. It never says a failing part is healthy."

Confidence is informative. Every wrong accumulator prediction had a confidence of 0.61 or less.

> "If we let the accumulator model say 'not sure' below 0.62 confidence, it hands just 9 of 441 cycles to an engineer. Those 9 include all 5 errors, so it is 100 % correct on the 432 cycles it does answer."

[MLapp, Robustness tab, selective prediction slider]

Noise robustness. We added Gaussian noise to every input, scaled to each feature's training standard deviation, and re-scored the test set.

Noise level	Cooler	Valve	Pump	Accumulator
None	1.000	1.000	1.000	0.988
0.1 × std	0.992	0.931	0.864	0.951
0.3 × std	0.904	0.754	0.642	0.822
1.0 × std	0.674	0.485	0.409	0.500

> "The pump model is the most fragile: a small amount of noise, one tenth of a standard deviation, already drops it to 0.86. It relies heavily on a few flow and motor-power features. The cooler holds up best at low noise. For a real plant this means sensor calibration and filtering matter."

These are macro-F1 scores, averaged over 3 random noise draws.

7. Advantages and drawbacks

Advantages

- **Accurate on this rig:** 0.988–1.000 macro-F1 on held-out test cycles, and 0.916–0.995 under the stricter blocked validation.
- **Fast:** one cycle is scored in well under a second, against a 60-second cycle.
- **Gives probabilities, not just labels:** confidence, margin and entropy show when a prediction is shaky, and low confidence reliably flagged every error.
- **Explainable:** the top features have physical meaning (cooling power, pressure variation, flow).
- **Handles imbalance:** class weights stop the models from ignoring rare fault levels.
- **Leakage-safe and reproducible:** one pipeline, fixed random seed 42, and the app never retrains.

Drawbacks: where the models fail

1. **Accumulator 90 vs 100 bar.** The only real confusion. The two lowest pre-charge levels look almost the same in the signals.
2. **Accumulator model is hard to explain.** It uses all 553 correlated features, so importance is spread thinly and permutation importance looks almost flat.
3. **Noise sensitivity.** The pump model falls to 0.86 macro-F1 with small noise (see section 6).

4. **Unstable cycles.** Two of the five errors (cycles 466 and 470) carry the dataset's 'stable flag' of 1, meaning the rig may not have settled.
5. **Closed set of classes.** A condition never seen in training, for example a cooler at 50 %, will still be forced into one of the known classes.
6. **One laboratory rig.** Faults were set on purpose at fixed levels, under one repeated load cycle. The models are not proven on other machines or on gradual real-world wear.
7. **Depends on virtual sensors.** The cooler relies on CE and CP, which the rig computes itself. A plant without them would need re-validation.
8. **No remaining useful life.** The data has condition levels, not failure times, so the models say what state a part is in, not when it will fail.

> "So the models are very good at recognising the fault levels this rig was built to produce. Their limits come mostly from the data, which I'll explain next."

8. Why the scores are so high

Expect this question. Answer it before they ask.

> "Scores of 0.99 and 1.00 should make anyone suspicious, including us. They are high for five reasons, and most of them are about the data, not about clever modelling."

1. **It is a controlled laboratory dataset.** Faults were set deliberately at a few discrete, well-separated levels, and the rig repeats the same 60-second load cycle every time. There is very little of the variation a real plant would add.
2. **Conditions were held for long runs.** In recording order the cooler changes level only **3 times**: each level is one continuous block of about 732 cycles. The accumulator changes 12 times, the pump 37 times, the valve about every 10 cycles.
3. **Neighbours are near-duplicates.** Because of point 2, almost every test cycle has a twin recorded seconds earlier in the training set. A random split rewards the model for recognising the twin. That is why blocked validation is lower, by up to 7.6 points.
4. **The cooler is almost read off directly.** Its classes line up with the virtual channels CP and CE, which measure cooling itself. The cooler blocks also coincide with slow drifts over the recording session, so part of its score may come from 'which session is this' rather than cooler physics.
5. **Good features make the classes separable.** After feature engineering, even a linear model reaches 0.99 on the valve.

> "So our honest summary is: near-perfect under a random split, and 0.92 to 0.995 when tested on unseen stretches of time."

9. Is this a perfect classifier?

Short answer: **no, and we should not claim it.**

> "A perfect classifier would label every future cycle correctly, from any machine and any operating condition. No finite test can prove that.
>
> In theory, the best any model can do is the **Bayes-optimal classifier**: for each input it picks the class that is truly most likely. If two classes genuinely overlap, like our 90 and 100 bar accumulator cycles, even the Bayes-optimal classifier makes some errors. Perfection is only possible when the classes do not overlap at all."

Three reasons 1.000 on our test set is not 'perfect':

1. **A finite sample.** Zero errors in 441 test cycles only shows the true error rate is probably small. By the 'rule of three', with 95 % confidence it could still be up to about 0.7 %.
2. **An optimistic split.** The test set shares near-duplicate neighbours with training, so it is an upper bound. Blocked validation gives 0.968 for the pump and 0.916 for the accumulator.
3. **One rig only.** Perfect on this rig says nothing yet about another machine.

Rule of three: upper bound on the error rate (95 % confidence) ≈ 3 / n = 3 / 441 ≈ 0.68 %.

> "So the correct phrasing is: the models are near-perfect on this laboratory dataset under a random split, with realistic performance between 0.92 and 0.995 macro-F1."

10. Closing and likely viva questions

> "To summarise: tree ensembles classify all four components with near-perfect scores on this rig. Their confidence tells us when they are unsure. The honest estimate on unseen stretches of time is 0.92 to 0.995 macro-F1, and the one real weakness is separating the two lowest accumulator pressures. Thank you."

Question	Answer in one or two lines
Why a random forest for the valve but XGBoost for the pump?	Both were within fold noise of each other. Each winner had the highest mean cross-validated macro-F1 on training data; we did not hand-pick.
Why macro-F1 instead of accuracy?	Three targets are imbalanced. Macro-F1 weights every class equally, so rare fault classes count as much as the healthy class.
What is data leakage and how did you prevent it?	Information from test data reaching training. Every learned step sits inside one Pipeline fitted per fold, and the test set was used once.
Why is test 100 % but blocked CV lower?	Neighbouring cycles are near-duplicates. Random splits put twins on both sides; blocked CV holds out whole time stretches.
How does class weighting work?	Each class gets weight N / (K × N_c), so a class with fewer cycles counts more when trees choose splits.
How is confidence computed?	It is the highest class probability: the forest's averaged tree votes, or XGBoost's softmax output.
What if a completely new fault appears?	The model will still pick one of its known classes. Low confidence is the warning sign; a real system would add an 'unknown' check.
Why not deep learning?	2,205 cycles is small for deep networks, and trees on engineered features already reach 0.99 and stay explainable.
Why fixed hyperparameters?	So differences come from model family and features, not tuning luck. Tuning is future work.
Can it predict when a part will fail?	No. The data has condition levels but no failure times, so remaining useful life is out of scope.